

Document Number: P0342R2

Date: 2023-04-11

Reply to: Gonzalo Brito Gadeschi <gonzalob_at_nvidia.com (<http://nvidia.com>)>

Authors: Gonzalo Brito Gadeschi, Mike Spertus

Audience: Concurrency, Library Evolution

pessimize_hint

This proposal adds an identity function, `std::pessimize_hint`, that *hints* the implementation to be maximally pessimistic in terms of the assumptions about what this function *could* do, i.e., to assume that it could do anything that well-defined C++ may do. It is useful for writing portable micro-benchmarks and teaching performance-related aspects of C++, which is quite valuable given how core *performance* is to C++'s value proposition as a programming language.

Motivation

Consider demonstrating the poor performance of the Fibonacci sequence as a teaching exercise:

```
#include<chrono>
#include<iostream>
using clk_t = std::chrono::steady_clock;
using dur_t = std::chrono::duration<double>;

size_t fib(size_t n) {
    if (n == 0) return n;
    if (n == 1) return 1;
    return fib(n - 1) + fib(n - 2);
}

int main() {
    auto start = clk_t::now();
    auto result = fib(42);
    auto elapsed = dur_t(clk_t::now() - start).count();
    std::cout << "Elapsed time is " << elapsed << " s" << std::endl;
    return 0;
}
```

Performance discussions should include data obtained by compiling with optimizations enabled, but it often requires a lot of trickery to produce “the right amount” of optimizations, which gets in the way of teaching C++. In this example, the compiler has fully optimized our micro-benchmark (godbolt (<https://godbolt.org/z/3a543nTW6>)):

```
call    std::chrono::_V2::steady_clock::now()@PLT
mov     rbx, rax
call    std::chrono::_V2::steady_clock::now()@PLT
```

The `std::pessimize_hint` API enables the programmer to hint the implementation their desire for the implementation to be maximally pessimistic when optimizing expressions consuming and producing certain values:

```
auto result = std::pessimize_hint(fib(std::pessimize_hint(42)));
```

Providing a simpler low-overhead way of achieving the desired pedagogical effect (godbolt (<https://godbolt.org/z/z77oachn6>)):

```

call    std::chrono::_V2::steady_clock::now()@PLT
mov     rbx, rax
mov     dword ptr [rsp + 4], 42
lea     rax, [rsp + 4]
movsxd  rdi, dword ptr [rsp + 4]
call    fib(unsigned long)
mov     qword ptr [rsp + 16], rax
lea     rax, [rsp + 16]
call    std::chrono::_V2::steady_clock::now()@PLT

```

A quality implementation will assume that the `expr` being pessimized, i.e., the call to `fib`, might do anything that well-defined C++ could do. Well-defined C++ could call `steady_clock::now()`, and therefore a quality implementation will ensure that it is sequenced before the second call `steady_clock::now()`.

This API has enabled us to construct a micro-benchmark for fibonacci implementations, that we can now include as part of our application's benchmarking suite, before starting to optimize the code.

For this purpose, micro-benchmarking libraries like Google Benchmark (<https://github.com/google/benchmark>) provides a `DoNotOptimize`

(<https://github.com/google/benchmark/blob/61497236ddc0d797a47ef612831fb6ab34dc5c9d/include/benchmark/benchmark.h#L306>) (similar to `pessimize_hint`). Programming languages with built-in language support for micro-benchmarking, like Rust, include APIs with similar semantics in its standard library (`core::hint::black_box` (https://doc.rust-lang.org/core/hint/fn.black_box.html)). Many examples showing how this API is used in actual Rust programs are available [here](https://docs.rs/releases/search?query=black_box) (https://docs.rs/releases/search?query=black_box).

History

At the 2016 Oulo meeting, Evolution Working Group reviewed R0 that proposed a solution to transparently provide correct behavior for clocks and provide standard library barriers for analogous problems. Due to implementability concerns, the sentiment was to not pursue that solution strategy further.

At the 2023 Issaqah meeting, SG1 reviewed R1 of the paper and suggested the author to pursue standardizing standard practice to solve the micro-benchmarking problem in an analogous way to the Prior Art covered below.

Proposed wording

Add this new function template to the General Utilities library (`<utility>`).

Add the following to `[utility.syn]` (<http://eel.is/c++draft/utility.syn>):

```

namespace std {
    // [utility.pessimize_hint]:
    template <typename T> T& pessimize_hint(T& t) noexcept;
    template <typename T> T const& pessimize_hint(T const& t) noexcept;
}

```

Then add a new `[utility.pessimize_hint]` sub-section containing:

```

template <typename T> T& pessimize_hint(T& t) noexcept;
template <typename T> T const& pessimize_hint(T const& t) noexcept;

```

- Preconditions: none.
- Mandates: nothing.
- Returns: a reference to the value passed in.
- Effects: none.
- Throws: nothing.
- [Note: implementations are encouraged to treat `pessimize_hint` as an *extern unknown function* that may perform any valid operation that well-defined C++ is allowed to perform.]

Discussion / FAQ

- Should we expose a way to control which optimizations to selectively disable?
 - The standard lacks vocabulary to talk about many optimizations, and adding this vocabulary would be non-trivial.
 - Some common requests are:
 - Enable / Disable FMA: `#pragma STDC FP_CONTRACT` already supports that.
 - Control inlining: `pessimize_hint(expr)` still allows function calls within the expression to be inlined. Some compilers have a `[[noinline]]` attribute that can be used at function definitions, declarations, and call-sites. This feature could stand on its own and therefore be pursued separately, enabling:
`pessimize_hint([[noinline]] foo())`.
 - Constant propagation: `pessimize_hint(x); foo(x, y);` can be used to block optimizations around `x` while still allowing optimizations on `y`.
- Does the suggestion that `pessimize_hint` should be treated as an “opaque” C++ function that can do anything that C++ can do extend to language extensions provided by the implementation?
 - Maybe, those are language extensions not covered by the standard, so that would be up to the implementation.
- Should this be in a different header?
 - Maybe. `<utility>` is not free standing.
- Does this proposal prevent the re-ordering of memory operations around `::now()` methods from `chrono` clocks?
 - No it does not.
- Does `pessimize_hint` work with expressions of type `void`?
 - No, it does not. For `void` expressions taking arguments, `pessimize_hint` can be used on the arguments. One can also capture the expression inside a lambda and pass it through `pessimize_hint`, e.g., given `void f()` and the `void` expression `f()`, one can do `pessimize_hint([] { f(); })();`.